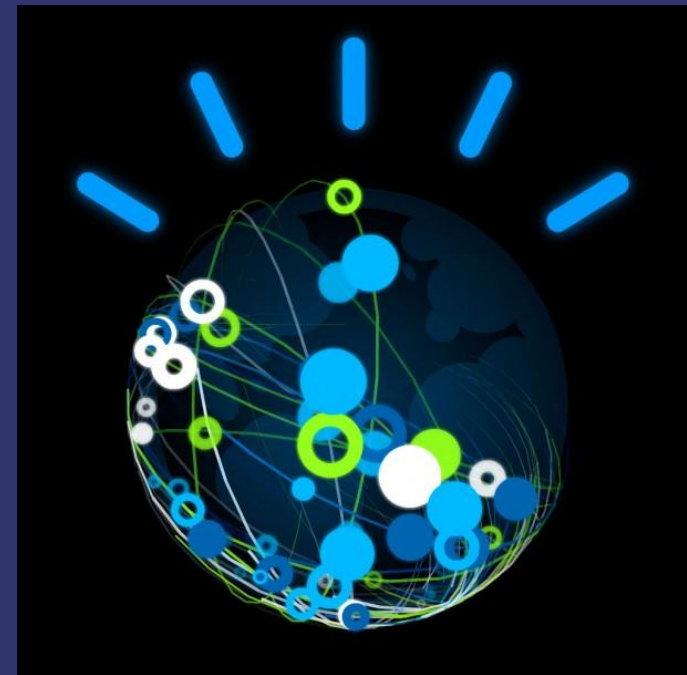


VEŠTAČKA INTELIGENCIJA, 2011/2012

PROGRAMSKI JEZICI VEŠTAČKE INTELIGENCIJE

Sadržaj

- Osnovne karakteristike
- Lisp
- Prolog



Prof. dr Leonid Stoimenov

Poredjenje konvencionalnih programa i VI

	Programi VI	Konv.programi
Vrsta obrade	simbolička	numerička
Model rešavanja	Deklarativan (deo baze znanja)	Proceduralan (algoritam)
Tip rešenja	Zadovoljavajuće	Optimalno
Upravljanje/podaci	Odvojeni	Isprepleteni
Znanje	Neprecizno	Precizno
Modifikacije	Česte	Retke

Osnovni koncepti kod sistema VI

- **Simboličko** umesto numeričkog izračunavanja.
- **Nealgoritamski pristup** rešavanju problema.
- **Zaključivanje** zasnovano na **znanju**.
- Primenljivost kod **loše struktuiranih problema i podataka**

Zašto posebni jezici za VI ?

- ❑ Složenost mnogih problema iz veštačke inteligencije
- ❑ Često se ne mogu primeniti principi za razvoj softvera iz softverskog inženjerstva
- ❑ Simbolička obrada
- ❑ Kod standardnih programskih jezika programer treba da vodi računa o velikom broju različitih struktura, tipova podataka, deklaraciji promenljivih, alokaciji memorije i sl.

Moderni programski jezici

- Primena principa softverskog inženjerstva
- Mogućnost primene alata i okruženja za razvoj
- ugrađene strukture podataka (liste, stabla i sl.)
- gotove operacije za pristup i manipulisanje (na primer, obilazak, ali i poklapanje uzoraka).
- Za realizaciju sistema VI:
 - ▣ C, C++, Java, ...

Jezici za simboličku obradu

- Prilagođeni problemima veštačke inteligencije
- koriste složene strukture podataka,
- kriju detalje (upravljanje memorijom, IO, dinamičke strukture)
- Nude viši nivo simboličke obrade koja ima svoju cenu – prevođenje i izvršavanje nije kao kod standardnih programskih jezika.

Dve paradigme

□ Funkcionalna,

- podrazumeva primenu rekurzivnih funkcija,
- glavni predstavnik ove grupe jezika je **Lisp**.

□ Logička

- primena principa logike predikata, odnosno principa formalne logike),
- glavni predstavnik ove grupe jezika je **Prolog**.

Lisp (LISt Processing)

- John McCarthy, 1958
- Jezik opšte namene
- Funkcionalan jezik (komponovanje funkcija i rekurzija)
- Interpretiran jezik (read-eval-execute)
- Sada takodje inkrementalna kompilacija
- Rekurzija

Osnovne osobine Lisp-a (1)

- Programi i podaci se tretiraju na **istovetan** način. Lisp koristi liste kao osnovne strukture podataka, a i poziv funkcije je lista čiji je prvi element ime funkcije (koristi se **prefix** notacija).
- Automatsko i dinamičko **upravljanje memorijom**. Sistem sam rezerviše i oslobađa memoriju prema potrebama.
- Jezik sa **funkcionalnim načinom** programiranja. Funkcionalno programiranje zasniva se na ideji da se rezultat svake funkcije može iskoristiti kao argument druge funkcije. Lisp omogućava komponovanje (kombinovanje) funkcija.
- **Rekurzija** je prirodni način programiranja.

Osnovne osobine Lisp-a (2)

- **Interpretiran** jezik. Primenjuje se ciklus *read-eval-execute*, Danas postoje i prevodioci za Lisp, a razvojna okruženja često nude i prevodioce i interpretatore. Obično postoji inkrementalno prevođenje.
- **Ne** postoje **tipovi** podataka. Lisp je programski jezik bez javne definicije tipa. Mnogi dijalekti nude mogućnost korišćenja tipova podataka, ali to nije neophodno. U tom slučaju provera i vezivanje tipa je dinamičko, tj. samo u procesu izvršenja.
- **Jednostavna sintaksa**. Lisp ima jednostavnu sintaksu, i može se primeniti kako za matematička izračunavanja, tako i za simboličku obradu.

Ispit iz VI i Lisp

- Common Lisp (Allegro CL, Franz inc.)
- <http://www.franz.com>
- Okruženje za razvoj
- Veliki broj ugradjenih funkcija
- Knjiga: **Common Lisp: the Language (Steele)**
- On-line dokumentacija
- Lab. vežbe se rade u Lisp-u
- Projekat 1 se radi u Lisp-u

Uvodne napomene o Lisp-u

- Umesto sekvencijalnog navođenja naredbi, Lisp koristi **ugnježdavanje i komponovanje funkcija** za definisanje programa.
- **Rekurzivno** definisanje funkcija, tako da program može imati jasno deklarativno značenje, bez opterećivanja nepotrebnim detaljima.
- Uniformnost, **odnosno** predstavljanje programa i podataka na isti način, preko osnovne strukture podataka - liste.
- Interaktivni jezik, tako da se najlakše uči na računaru.

Osnovni elementi jezika Lisp

S-izraz

- **Atom** (simbol, broj, string, t, nil, ...)
 - ▣ Nema razlike izmedju malih i velikih slova (osim u stringovima)
- **Lista**: elementi liste su atomi ili liste
 - ▣ Prvi element liste se tretira kao ime funkcije, ako ispred liste nije *quote* (‘)

Primer: Liste u Lisp-u

Atomi

- A 1 5 neki-atom_sa_dugackim-imenom

Liste:

- (A B C)
- (1 b a 3 c)

Lista atoma:

- (1 2 3 4 a b)

Prava lista (lista listi):

- (1 (2 3) (c d (e) f) 4 5)

Funkcije

- **Funkcije** se zadaju u prefiks notaciji, takođe u obliku liste, tako da je prvi element liste ime funkcije, a ostali elementi argumenti. Za poziv funkcije navodi se ime funkcije iza koga slede argumenti:

(ime-funkcije {argumenti})

Primer:

(+ 2 3 5 7 100)

Razlika između funkcija i podataka

- `'(obj1 obj2 obj3 ... objn)` - lista kao podatak
- `quote(obj1 obj2 obj3 ... objn)` - lista kao podatak
- `(proc arg1 arg2 ... argn)` - poziv funkcije

Primer: Razlika između poziva funkcija i liste

- Liste (podatak):
- `'(+ 9 3)` ili `(quote (+ 9 3))`
- Poziv funkcije:
- `(+ 9 3) => 12`

Konstante i specijalne vrednosti

- “string”
- 1, 0, 5
- 'simbol
- #\a (karakter)
- #(1 2 3) (vektor)
- TRUE i FALSE
 - FALSE: (), NIL ili nil

Sistemske funkcije

- Numericke: $+$, $-$, $*$, $/$, $\log$, ...
- Logicke: *and*, *or*, *not*, *every*
- Funkcije za rad sa listama: *cons*, *list*, *append*, *car*, *cdr*, *first*, *rest*, *nth*, *length*, ...
- Predikati: *atom*, *consp*, *listp*, $<$, $=$, *equal*, *member*, ...

Primer: Numeričke funkcije

- $(+ 1 2 3 4 5) \Rightarrow 15$
- $(- 1 2 3 4 5) \Rightarrow -13$
- $(* 5 4 3) \Rightarrow 60$
- $(/ 12 3 2) \Rightarrow 2$
- $(\text{random } 100) \Rightarrow 55$
- $(\text{sqrt } 2) \Rightarrow 1.4142135$
- $(\text{exp } 1) \Rightarrow 2.7182817$
- $(\text{square } 5) \Rightarrow 25$

Primer: Logičke funkcije

- (not obj) - negacija objekta *obj*
- (and obj1 obj2 ... objn) - logičko I objekata *obji*
- (or obj1 obj2 ... objn) - logičko IЛИ objekata *obji*

- (and '() '(1 2 3)) => ()
- (or '() '(1 2 3)) => (1 2 3)

Funkcije za rad sa listama

- *cons*
 - *list*
 - *append*
 - *car*
 - *cdr*
 - *first*
 - *rest*
 - *length*, itd.
- Primer:
 - $(\text{car } '(1\ 2\ 3\ 4)) \Rightarrow 1$ - prvi element liste
 - $(\text{cdr } '(1\ 2\ 3\ 4)) \Rightarrow (2\ 3\ 4)$ - ostatak liste, tj. svi elementi kao lista, osim prvog
 - $(\text{first } '(1\ 2\ 3\ 4)) \Rightarrow 1$ - prvi element liste
 - $(\text{rest } '(1\ 2\ 3\ 4)) \Rightarrow (2\ 3\ 4)$ - ostatak liste, tj. svi elementi kao lista, osim prvog

Upravljačke funkcije

- Defun, defparamater, defvar
(*defun kvadrat(n) (* n n)*)
- Uslovne: *if*, *cond*, *when*, *unless*
(*defun fact(n)*
 (*if (= n 0) 1 (* n (fact (1 - n)))*))
- Iteracija: *do*, *do**, *dolist*
- Lokalne promenljive: *let* i *let**
- Globalne promenljive: *setf*
- *Eval*, *funcall*, *apply*
- U/I funkcije: *open*, *close*, *with-open-file*, *read*, *write*, ...

Funkcija COND

- Funkcija sa proizvoljnim brojem argumenata
- Svaki argument sadrži dva S-izraza:
 - ▣ Uslov
 - ▣ Rezultat

(COND

(condition1 result1)

(condition2 result2)

...

(T resultN))

Struktura programa

- Program = Forme + Funkcije
- Forme mogu da budu: samo-evaluirajuće (self-evaluating), simboli (symbols, variables), liste (Lists)
- Argumenti funkcija mogu biti konstante, atomi ili liste, ali takođe i pozivi drugih funkcija.

(DEFUN *function_name parameter_list*
function_body)

Jednostavniji primeri (1)

□ (defun kvadrat(n)
 (* n n))

=> KVADRAT

□ Poziv realizovane funkcije:

(kvadrat 3) => 9

(kvadrat 5) => 25

Jednostavniji primeri (2)

- (defun rotate-left (l)
 (append (cdr l)
 (list (car l))))

- Poziv funkcije:

(rotate-left '(a b c d e f))

=> (B C D E F A)

Jednostavniji primeri (3)

```
(defun flatten (l)
  (cond
    ((atom l) l)
    ((atom (car l)) (cons (car l) (flatten (cdr l))))
    (:else (append (flatten (car l)) (flatten (cdr l)))) ) )
```

Primeri poziva:

(flatten '(1 2 3)) => (1 2 3)

(flatten '(1 2 (3 4) 5)) => (1 2 3 4 5)

„Logika“ Lisp programiranja

- LISP radi sa listama! Pošto listu obrađujete tako što obrađujete element po element liste, većina funkcija ima sličnu strukturu!
- Za početak, dovoljno je da sledite sledeća pravila:
- **Rule 1: Počnite sa COND.**
 - ▣ Većina funkcija sadrži COND. Ostatak se sastoji u navođenju svih slučajeva zadatih uslovom.
- **Rule 2: Test na NULL (praznu) listu.**
 - ▣ Prvi slučaj je najjednostavniji uslov, za izlazak iz rekurzije! Čak 90% funkcija koje obrađuju listu imaju kao počeni uslov test na NULL, i rade direktno sa praznom listom.

... nastavak

- ❑ **Rule 3: Uradite nešto sa CAR, zatim rekurzivno ostatak liste sa CDR.**
 - ❑ Drugi uslov – ako lista nije prazna, obrađujete prvi element (CAR) i rekurzivno pozivate istu funkciju za ostatak liste (CDR).
 - ❑ Obrada može da bude i kompleksna!!
- ❑ **Rule 3a: Ako vam ne treba CAR, jednostavno ga ignorišete i nastavljate sa CDR.**
- ❑ **Rule 3b: Da bi sačuvali CAR, koristite CONS na listu dobijenu rekurzivnim pozivom za CDR.**

... nastavak

- Primer: Brisanje svih atoma iz liste

```
(DEFUN REMATOMS (L)
```

```
  (COND ((NULL L) L)
```

```
        ((ATOM (CAR L)) (REMATOMS (CDR L)))
```

```
        (T (CONS (CAR L) (REMATOMS (CDR L)))) ) )
```

- **Rule 3c: Za transformaciju elemenata liste, uradite CONS transformisanog CAR na rezultat rekurzivnog poziva za CDR.**

... nastavak

- Primer: dodati 1 svakom elementu liste brojeva (koristiti 1 + funkciju):

```
(DEFUN ADDONE (L)
```

```
  (COND ((NULL L) L)
```

```
        (T (CONS (1 + (CAR L)) (ADDONE (CDR L))))))
```

- **Rule 4: Kod COND koristite to da su svi prethodni uslovi neispunjeni.**
 - Vodite računa o legalnosti poziva! Na primer, ne pozivajte (CAR L) ako znate da je L prazna lista, ili ne pozivajte (EQ X Y) dok niste sigurni da su X i Y atomi.

... nastavak

- ❑ **Rule 5: Koristite T kao poslednji uslov u COND.**
 - ❑ COND treba da vam pokriva sve slučajeve, i da vrati vrednost za svaki od uslova ako je ispunjen.
- ❑ Umesto zaključka:
 - ❑ programeri u klasičnim algoritamskim programskim jezicima mogu da imaju probleme kod programiranja u Lisp-u.
 - ❑ Lisp nudi naredbe za standardne upravljačke strukture, ali je preporuka da ih ne koristite
 - ❑ Lisp koristi kompoziciju funkcija i rekurziju

LISP – dodatne mogućnosti

- ❑ Paketi (packages)
- ❑ Struktura
- ❑ CLOS: objektno-orijentisana ekstenzija

PROLOG



- PROgramming in LOGic
- Deklarativan jezik
- Jednostavna sintaksa: aksiomi, pravila i pitanja
- Rekurzija

PROLOG - nastavak

- Logika predikata prvog reda
- Deklarativno programiranje
- Atomi (konstante): brojevi, simboli, promenljive
- Promenljive (pocinju velikim slovom ili _)
 _*x*
- Strukture: skupovi entiteta, predstavljaju relacije izmedju objekata
 prijatelj(tigar, medved).
 prijatelj(tigar,X).
- Izrazi

PROLOG - nastavak

- Pravila *zaključak:- konjukcija premisa*
- Uvek su univerzalno kvantifikovana
cist(X):- opran(X).
opran(X):- nasapunjan(X), ispran(X).
- Traženje *?-*
?- y=1+2 daje y=1+2;
?- y is 1+2 daje y=3;
?- 1+2 ::= 2+1 daje Yes

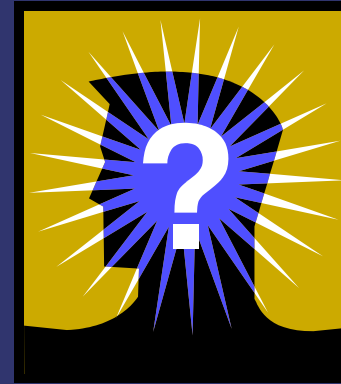
PROLOG - nastavak

□ Disjunkcija

Mrtav(X) :- nema_puls(X).

Mrtav(X) :- ne_dise(X).

PITANJA?



Dileme?

Komentari?

